

Hands-on Activity 7.1	
Sorting Algorithms Pt1	
Course Code: CPE010	Program: Computer Engineering
Course Title: Data Structures and Algorithms	Date Performed: Sep 18, 2025
Section: CPE21S4	Date Submitted: Sep 18, 2025
Name(s): Punay, Heidee S.	Instructor: Engr. Jimlord Quejado

6. Output

	<pre>const int SIZE = 100; void printArr(const int arr[], int size) { for (int i = 0; i < size; i++) { std::cout << arr[i] << " "; if ((i + 1) % 20 == 0) std::cout << "\n"; // break line every 20 elements } std::cout << "\n"; }</pre>
Observation	firstly was to define the size of the array which is the const size of 100. Made a function where the parameter to an array that is to be printed and the number of elements in the array which is 100. Inside the function is a for loop where the loop runs from 0 to 99 times. The if statement is for adding a newline every 20 elements.

	<pre>template <typename T> void bubbleSort(T arr[], size_t arrSize){ for(int i= 0; i < arrSize; i++){ for(int j = i+1; j < arrSize; j++){ if(arr[j] > arr[i]){ std::swap(a: [&] arr[j], b: [&] arr[i]); } } } }; bubbleSort(bubbleArr, SIZE); std::cout << "Bubble Sorted Array: "<<std::endl; printArr(bubbleArr, SIZE);</pre>
--	---

Observation	Made an template to sort arrays in any data type and after, it creates a function for
-------------	---

bubble sort where its parameter is the array and the size of the array. Outer loop goes through each element and the inner loop compares the arr[i] and if the arr[j] is greater than arr[i], they swap.

```
template <typename T> int Routine_Smallest(T A[], int K, const int arrSize);
template <typename T>
void selectionSort(T arr[], const int n) {
    int POS, temp, pass = 0;
    for (int i = 0; i < n; i++) {
        POS = Routine_Smallest(arr, K: i, arrSize: n);
        temp = arr[i];

        arr[i] = arr[POS];
        arr[POS] = temp;

        pass++;
    }
}
```

```
selectionSort(selectionArr, n: SIZE);
std::cout << "Selection Sorted Array: "<<std::endl;
printArr(selectionArr, SIZE);
```

Observation

This sorts the array in ascending order, starting from smallest to biggest number. A template for any data types in an array and a function where the array is to be sorted and also the number of elements in the array. A loop inside the function that goes through the array where pos stores the number of the smallest element found, temp to hold the value temporarily during swapping. It finds the smallest element from i until it finds the smallest element using the Routine_smallest and next is when the smallest element is found, the arr[i] swaps with the smallest element arr[POS] and repeats until all the elements are sorted.

```
//insertion sort
template <typename T>
void insertionSort(T arr[], const int n) {
    int K = 0, j, temp;
    while (K < n) {
        temp = arr[K];
        j = K-1;
        while (temp <= arr[j]) {
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = temp;
        K++;
    }
}
```

```
insertionSort(insertionArr, n:SIZE);
std::cout << "Insertion Sorted Array: " << std::endl;
printArr(insertionArr, SIZE);
```

Observation

Made a function for insertion sort where it is the array and the number of elements in the array. Inside the function is a K where is the current element that will be sorted into the sorted portion. A while loop that loops through the element in the array and a temp where the K will be stored and then a j that compares the previous element. A while loop again that if the current element is less than the previous element, it shifts the previous value one position to the right and move the j backward. And it inserts the current value to its correct position and moves to the next element.

cpp file

```
#include <iostream>
#include <cstdlib> // for rand()
#include "sort.h"

const int SIZE = 100;

void printArr(const int arr[], int size) {
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
        if ((i + 1) % 20 == 0) std::cout << "\n"; // break line every 20 elements
    }
    std::cout << "\n";
}
```

```

int main() {
    int original[SIZE];
    for (int i = 0; i < SIZE; i++) {
        original[i] = rand() % 100;
    }

    // Create copies for each sort
    int bubbleArr[SIZE], selectionArr[SIZE], insertionArr[SIZE];
    std::copy(original, original + SIZE, bubbleArr);
    std::copy(original, original + SIZE, selectionArr);
    std::copy(original, original + SIZE, insertionArr);

    std::cout << "Original Unsorted Array: " << std::endl;
    printArr(original, SIZE);

    bubbleSort(bubbleArr, SIZE);
    std::cout << "Bubble Sorted Array: " << std::endl;
    printArr(bubbleArr, SIZE);

    selectionSort(selectionArr, SIZE);
    std::cout << "Selection Sorted Array: " << std::endl;
    printArr(selectionArr, SIZE);

    insertionSort(insertionArr, SIZE);
    std::cout << "Insertion Sorted Array: " << std::endl;
    printArr(insertionArr, SIZE);

    return 0;
}

```

header

```

#ifndef DSA_SORT_H
#define DSA_SORT_H
#include <iostream>

// bubble sort
template <typename T>
void bubbleSort(T arr[], size_t arrSize){
    for(int i= 0; i < arrSize; i++){
        for(int j = i+1; j < arrSize; j++){
            if(arr[j] > arr[i]){
                std::swap(arr[j], arr[i]);
            }
        }
    }
};

//selection sort (array, size of array)
template <typename T> int Routine_Smallest(T A[], int K, const int arrSize);
template <typename T>
void selectionSort(T arr[], const int n) {
    int POS, temp, pass = 0;
    for (int i = 0; i < n; i++) {
        POS = Routine_Smallest(arr, i, n);
        temp = arr[i];

        arr[i] = arr[POS];
        arr[POS] = temp;

        pass++;
    }
}

```

```

    }
}

// routine smallest (array, current position, size of array)
template <typename T>
int Routine_Smallest(T A[], int K, const int arrSize) {
    int position, j;
    T smallestElem = A[K];
    position = K;
    for (int j= K+1; j < arrSize; j++) {
        if (A[j] < smallestElem) {
            smallestElem = A[j];
            position = j;
        }
    }
    return position;
}

//insertion sort
template <typename T>
void insertionSort(T arr[], const int n) {
    int K = 0, j, temp;
    while (K < n) {
        temp = arr[K];
        j = K-1;
        while (temp <= arr[j]) {
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = temp;
        K++;
    }
}
#endif //DSA_SORT_H

```

7. Supplementary Activity

```

#ifndef DSA_VOTE_SORT_H
#define DSA_VOTE_SORT_H
#include <iostream>

// bubble sort descending order
template <typename T>
void bubbleSort(T arr[], size_t arrSize) {
    for (int i = 0; i < arrSize; i++) {
        for (int j = i + 1; j < arrSize; j++) {
            if (arr[j] > arr[i]) {
                std::swap(arr[j], arr[i]);
            }
        }
    }
}

#endif

```

```

#include <iostream>
#include <cstdlib> // for rand()
#include "vote_sort.h"

const int SIZE = 100;

```

```

void printArray(const int arr[], int size) {
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
        if ((i + 1) % 20 == 0) std::cout << " " << std::endl;
    }
    std::cout << "\n";
}

// Manual array copy
void copyArray(const int source[], int destination[], int size) {
    for (int i = 0; i < size; i++) {
        destination[i] = source[i];
    }
}

int main() {
    int original[SIZE];
    int bubbleArr[SIZE];

    for (int i = 0; i < SIZE; i++) {
        original[i] = rand() % 100;
    }

    copyArray(original, bubbleArr, SIZE);

    std::cout << "Original Unsorted Array: " << std::endl;
    printArray(original, SIZE);

    bubbleSort(bubbleArr, SIZE);
    std::cout << "Bubble Sorted Array (Descending): " << std::endl;
    printArray(bubbleArr, SIZE);

    return 0;
}

```

Original Unsorted Array:

```
41 67 34 0 69 24 78 58 62 64 5 45 81 27 61 91 95 42 27 36
91 4 2 53 92 82 21 16 18 95 47 26 71 38 69 12 67 99 35 94
3 11 22 33 73 64 41 11 53 68 47 44 62 57 37 59 23 41 29 78
16 35 90 42 88 6 40 42 64 48 46 5 90 29 70 50 6 1 93 48
29 23 84 54 56 40 66 76 31 8 44 39 26 23 37 38 18 82 29 41
```

Bubble Sorted Array:

```
99 95 95 94 93 92 91 91 90 90 88 84 82 82 81 78 78 76 73 71
70 69 69 68 67 67 66 64 64 64 62 62 61 59 58 57 56 54 53 53
50 48 48 47 47 46 45 44 44 42 42 42 41 41 41 41 40 40 39 38
38 37 37 36 35 35 34 33 31 29 29 29 29 27 27 26 26 24 23 23
23 22 21 18 18 16 16 12 11 11 8 6 6 5 5 4 3 2 1 0
```

Selection Sorted Array:

```
0 1 2 3 4 5 5 6 6 8 11 11 12 16 16 18 18 21 22 23
23 23 24 26 26 27 27 29 29 29 29 31 33 34 35 35 36 37 37 38
38 39 40 40 41 41 41 41 42 42 42 44 44 45 46 47 47 48 48 50
53 53 54 56 57 58 59 61 62 62 64 64 64 66 67 67 68 69 69 70
71 73 76 78 78 81 82 82 84 88 90 90 91 91 92 93 94 95 95 99
```

8. Conclusion

To conclude, I learned much in this activity, like how are the loops works in sorting algorithms; either for loop and while loop where they go through the array and thats where it adds the different statements made. Finally, I can create a source file and a main file because I was having a hard time making one in the previous activities. The different ways of sorting algorithms: bubble sort, selection sort, and insertion sort where they have the same output though the bubble sort is in this descending order.

In the procedure, where all the sort were created: bubble sort, selection sort, and insertion sort. From what I have understood. First was too define the file create a function to each sort. The typename is for the array to sort any data types and after it was to create a frst function which is for the bubble sort where the parameter of it is the array and the size of the array it is almost the same with the other sort though the difference is just instead of the size others are the number of elements in the array. The outside loop where it goes through element in the array, if i is less than the size it goes through each element. Another loop, the inside loop compares the element i and j where if the j is greater than. That goes the same with selection and insertion, it compares and a statement that it will swaps the element to sort it.

9. Assessment Rubric